PORTFOLIO DEVELOPMENT ACTIVITY

Dynamic Portfolio Section: Projects CRUD

Project Documentation and Validation Evidence

Name: Nelmar Penera

Course: BS Information Technology - 3rd Year

Section: Not provided

Date: November 3, 2026

Review before submission: Course section, original Part 2/Part 4 responses, personalized reflections, and video demonstration link were not provided. Draft answers and a proposed validation table are included and labeled for editing.

1. Section Chosen and Justification

Chosen section: Projects / Featured Projects.

> **Draft answer to personalize:** I chose Projects because it is a core part of a portfolio and each project has several useful details, including its title, category, description, technologies, completion date, and links. Storing these records in the database lets me manage them through CRUD pages and show the current project list on the public portfolio without editing static page content each time.

2. Screenshots of All Pages

Figures 1-7 show the requested public, management, confirmation, success, and validation states. A temporary example record was created only for the screenshots and then deleted.

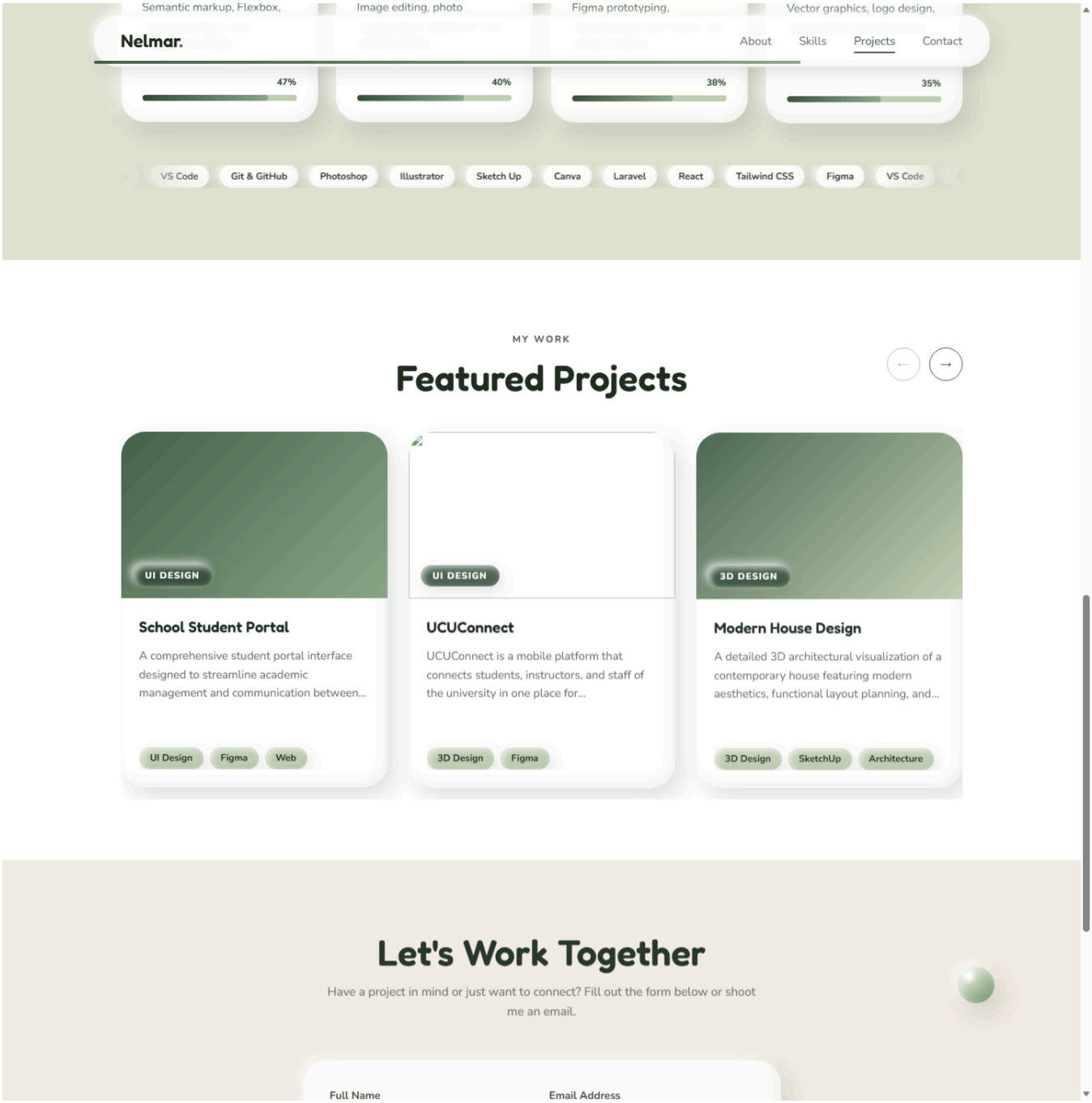


Figure 1. Public portfolio page showing the database-backed Featured Projects section.

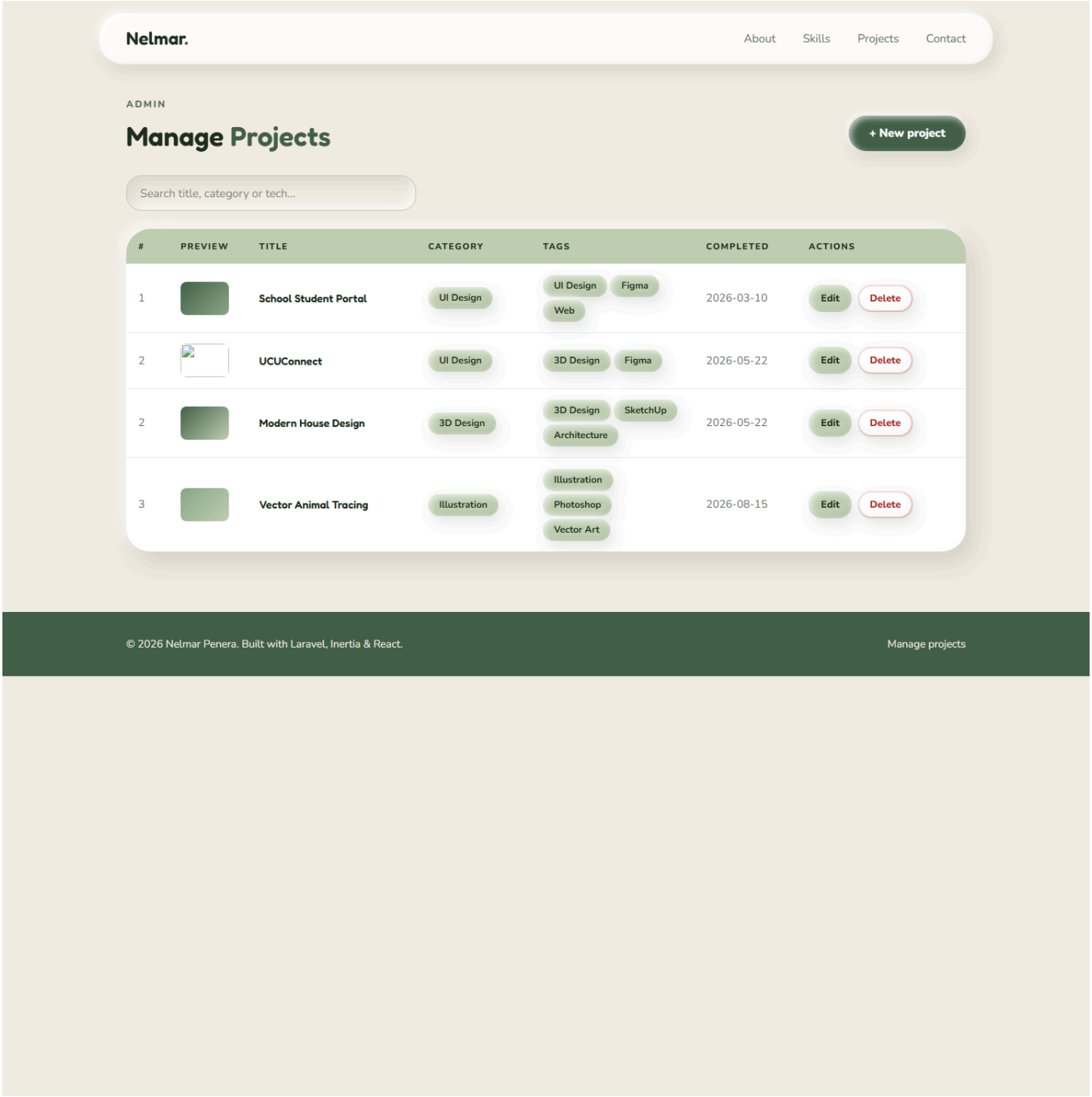


Figure 2. Projects index page listing the records.

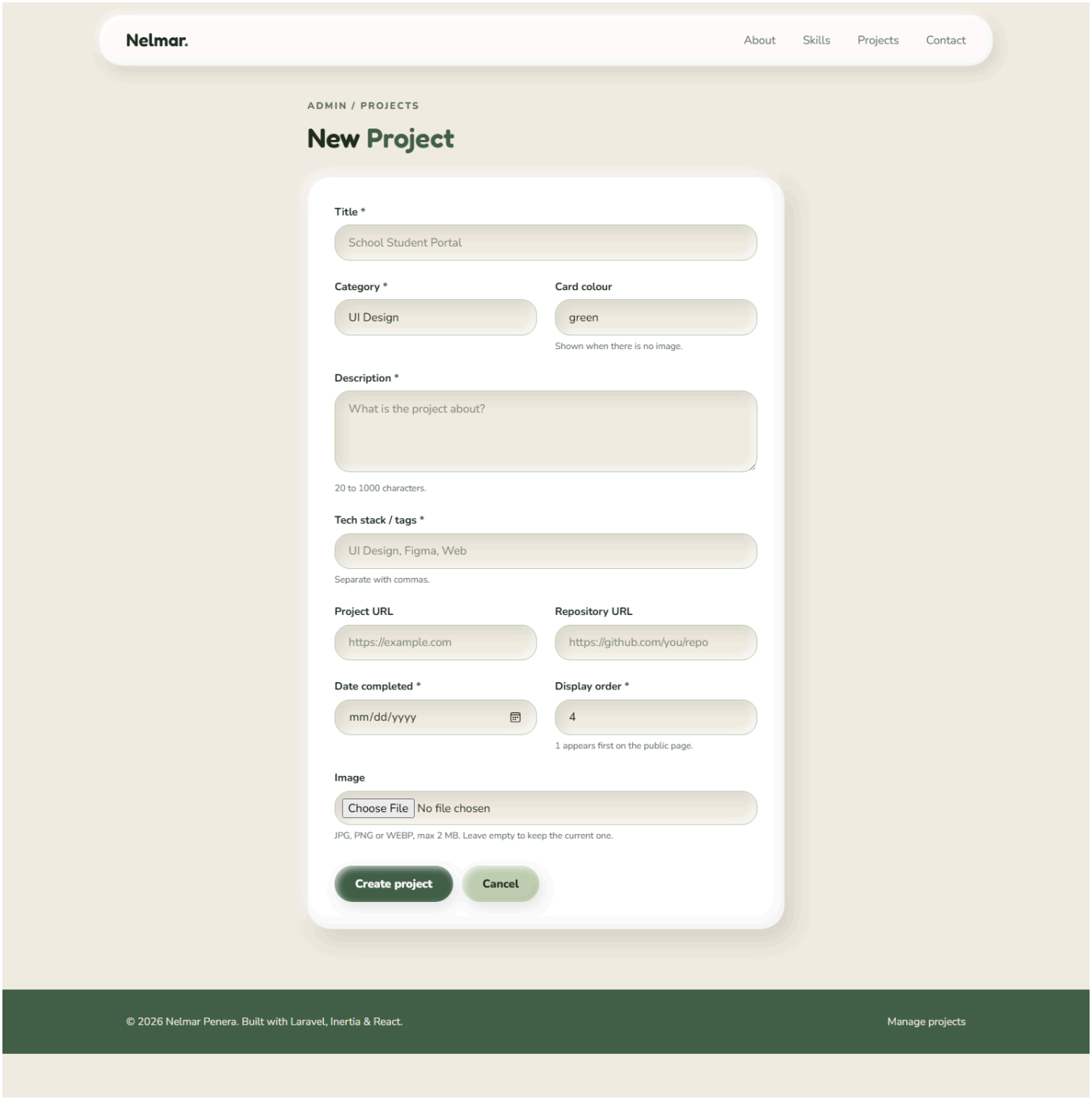


Figure 3. Create page for a new project.

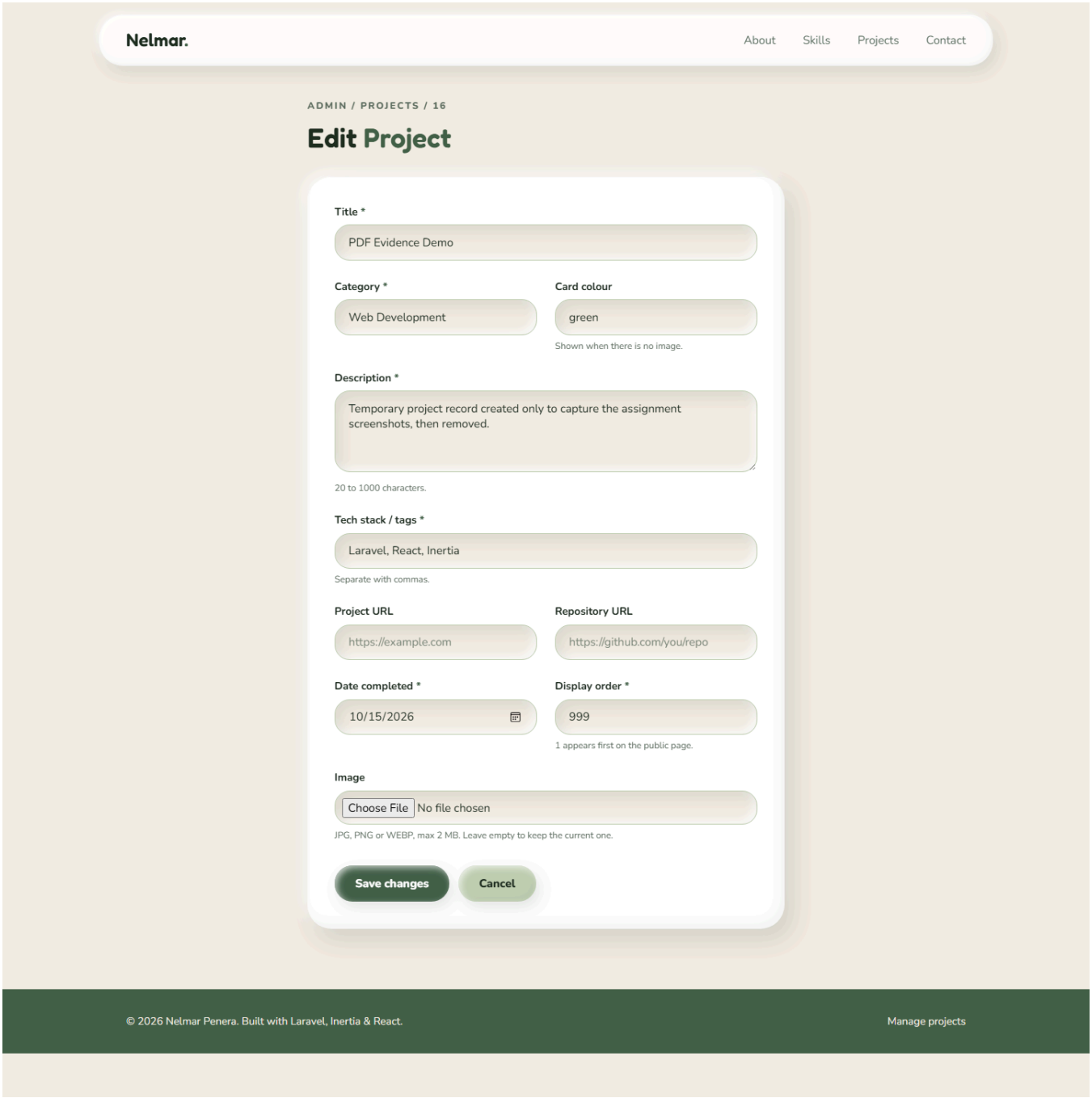


Figure 4. Edit page with a project loaded into the form.

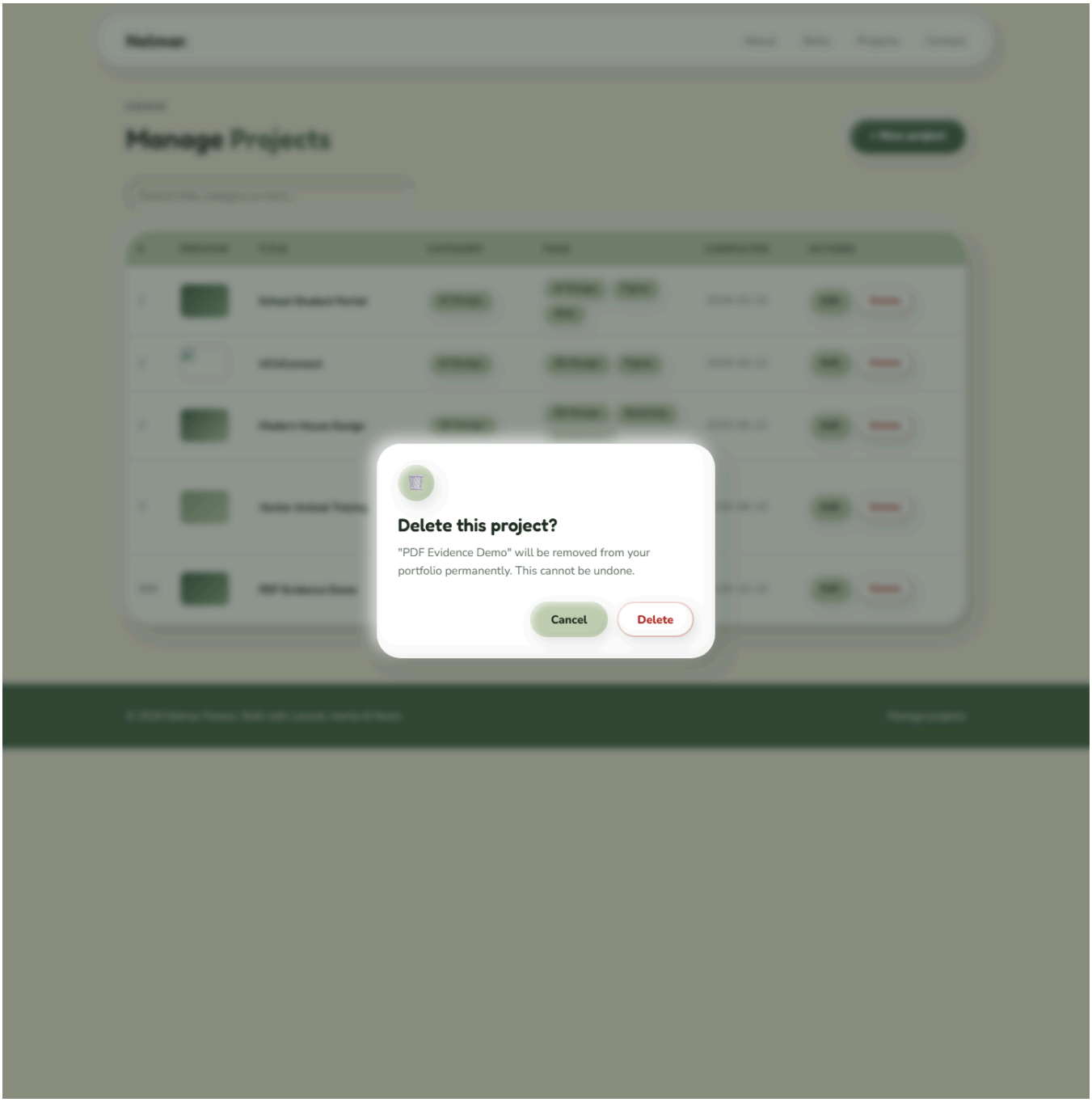


Figure 5. Delete confirmation dialog before removing the temporary screenshot record.

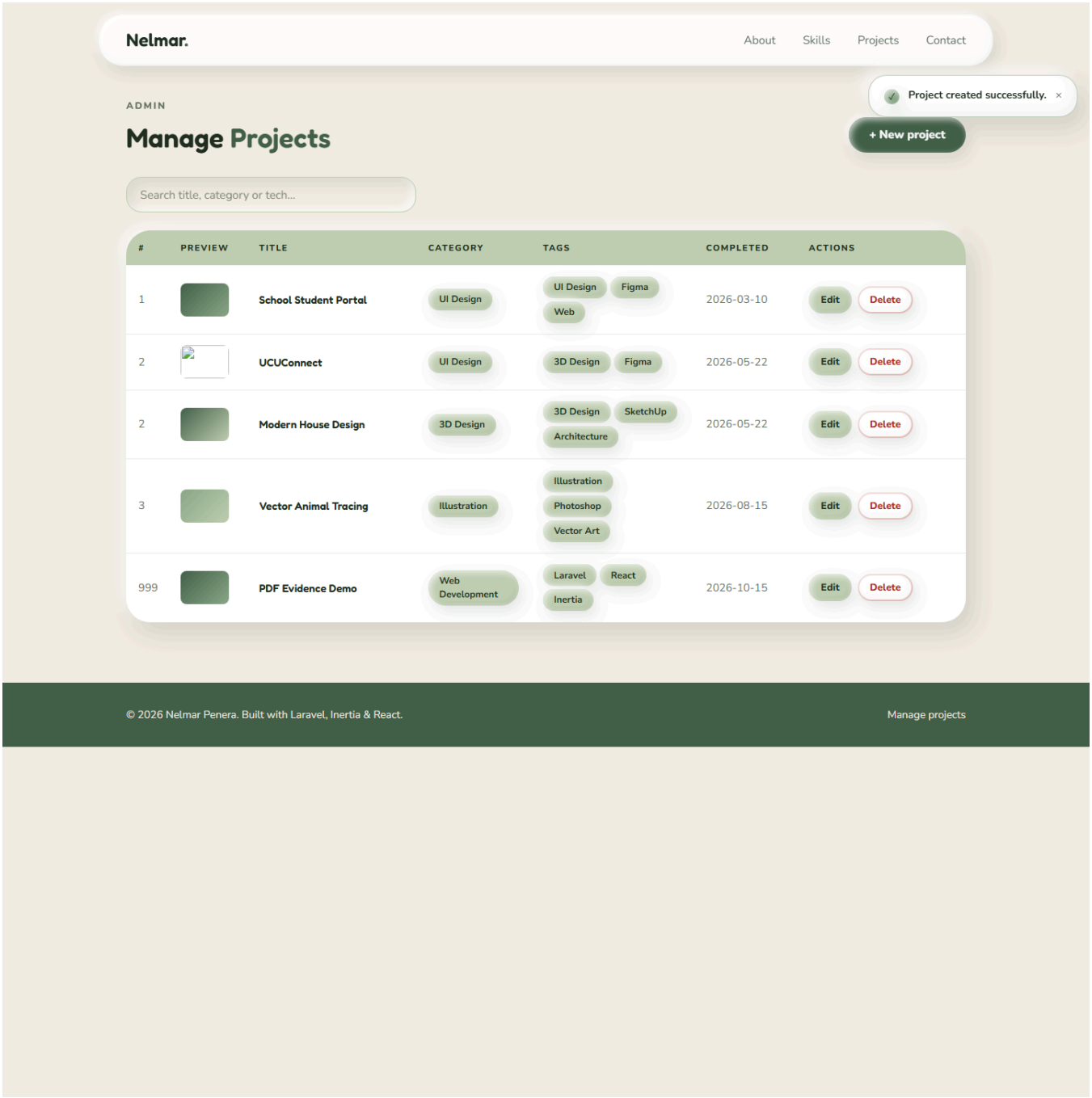


Figure 6. Success notification after creating the temporary screenshot record.

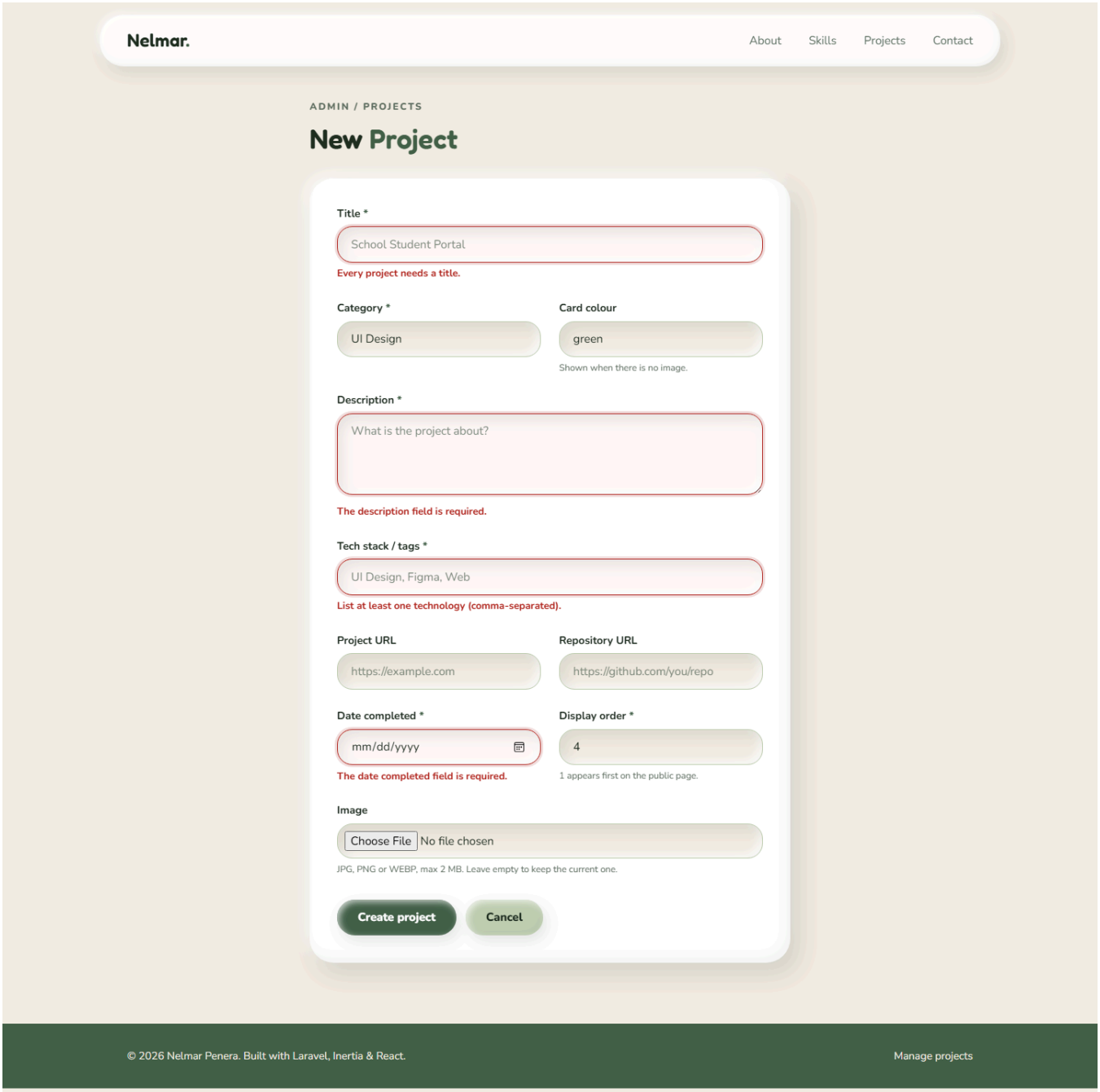


Figure 7. Required-field validation errors displayed on the form.

3. Folder Structure

The tree below lists the files included in the source appendix.

Projects Section - Folder Structure

```
portfolioninelmar/
+-- app/
|   +-- Http/Controllers/ProjectController.php
|   +-- Models/Project.php
+-- database/migrations/
|   +-- 2026_10_26_000000_create_projects_table.php
+-- routes/web.php
+-- resources/js/
    +-- Pages/
    |   +-- Projects/
    |   |   +-- Index.jsx
    |   |   +-- Create.jsx
    |   |   +-- Edit.jsx
    |   +-- Portfolio.jsx
    +-- Components/
        +-- ProjectForm.jsx
        +-- Field.jsx
        +-- ConfirmModal.jsx
        +-- FlashMessage.jsx
        +-- ProjectCard.jsx
        +-- Navbar.jsx
```

Paths shown are the files included in the source-code appendix.

Figure 8. Folder structure of the project files shown in the source appendix.

4. Source Code

Each appendix below contains the complete current file contents and a short explanation. Explanations are first-person drafts to review and personalize.

1. A. Database Migration - `database/migrations/2026_10_26_000000_create_projects_table.php`
2. B. Eloquent Model - `app/Models/Project.php`
3. C. Resource Controller - `app/Http/Controllers/ProjectController.php`
4. D. Web Routes - `routes/web.php`
5. E1. React Index Page - `resources/js/Pages/Projects/Index.jsx`
6. E2. React Create Page - `resources/js/Pages/Projects/Create.jsx`
7. E3. React Edit Page - `resources/js/Pages/Projects/Edit.jsx`
8. E4. Public Portfolio Page - `resources/js/Pages/Portfolio.jsx`
9. F1. Shared Project Form - `resources/js/Components/ProjectForm.jsx`
10. F2. Field Component - `resources/js/Components/Field.jsx`
11. F3. Delete Confirmation - `resources/js/Components/ConfirmModal.jsx`
12. F4. Success Message - `resources/js/Components/FlashMessage.jsx`
13. F5. Public Project Card - `resources/js/Components/ProjectCard.jsx`
14. F6. Navbar Component - `resources/js/Components/Navbar.jsx`

A. Database Migration

database/migrations/2026_10_26_000000_create_projects_table.php

I use this migration to create the projects table and define the fields the portfolio needs. It also makes project titles unique and records the ordering and completion date used by the public page.

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('projects', function (Blueprint $table) {
            $table->id();
            $table->string('title', 100)->unique();         // project name; unique so two cards never share a title
            $table->string('category', 50);                  // badge on the card, e.g. "UI Design"
            $table->string('gradient', 20)->default('green'); // card colour used when there is no image
            $table->text('description');                     // what the project is about
            $table->string('tech_stack');                    // comma-separated tags, e.g. "UI Design, Figma, Web"
            $table->string('project_url')->nullable();       // live link
            $table->string('repository_url')->nullable();    // GitHub link
            $table->string('image')->nullable();             // path on the public disk
            $table->date('date_completed');                  // when it was finished
            $table->unsignedSmallInteger('display_order')->default(1); // order on the public page
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('projects');
    }
};
```

B. Eloquent Model

app/Models/Project.php

I use the Project model as the shared definition of project data. It limits mass assignment to known columns and formats image URLs, tags, dates, and ordering values for the React interface.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Support\Facades\Storage;

class Project extends Model
{
    // Single source of truth for the category list (used by validation AND sent to the React form).
    public const CATEGORIES = ['UI Design', '3D Design', 'Illustration', 'Web Development', 'Branding', 'Other'];
    public const GRADIENTS = ['green', 'purple', 'amber', 'blue', 'rose'];

    // Only these columns can be mass-assigned via Project::create() / ->update().
    protected $fillable = [
        'title',
        'category',
        'gradient',
        'description',
        'tech_stack',
        'project_url',
        'repository_url',
        'image',
        'date_completed',
        'display_order',
    ];

    // Extra computed values that are sent to React with every record.
    protected $appends = ['image_url', 'tech_list'];

    protected function casts(): array
    {
        return [
            'date_completed' => 'date:Y-m-d',
            'display_order' => 'integer',
        ];
    }

    public function getImageUrlAttribute(): ?string
    {
        return $this->image ? Storage::disk('public')->url($this->image) : null;
    }

    public function getTechListAttribute(): array
    {
        return array_values(array_filter(array_map('trim', explode(',', (string) $this->tech_stack))));
    }
}
```

C. Resource Controller

app/Http/Controllers/ProjectController.php

I use this controller for the list, create, update, and delete workflows. It validates input with shared rules and custom messages, manages uploaded images, and flashes a success message after each change.

```
<?php

namespace App\Http\Controllers;

use App\Models\Project;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;
use Illuminate\Validation\Rule;
use Inertia\Inertia;
use Inertia\Response;

class ProjectController extends Controller
{
    /** List all project records. */
    public function index(): Response
    {
        $projects = Project::orderBy('display_order')
            ->orderByDesc('date_completed')
            ->get();

        return Inertia::render('Projects/Index', [
            'projects' => $projects,
        ]);
    }

    public function create(): Response
    {
        return Inertia::render('Projects/Create', [
            'categories' => Project::CATEGORIES,
            'gradients' => Project::GRADIENTS,
            // pre-fill display order with the next free number
            'nextOrder' => ((int) Project::max('display_order')) + 1,
        ]);
    }

    public function store(Request $request): RedirectResponse
    {
        $validated = $request->validate($this->rules(), $this->messages());

        if ($request->hasFile('image')) {
            $validated['image'] = $request->file('image')->store('projects', 'public');
        }

        Project::create($validated);

        return redirect()->route('projects.index')
            ->with('success', 'Project created successfully.');
    }

    public function edit(Project $project): Response
    {
        return Inertia::render('Projects/Edit', [
            'project' => $project,
            'categories' => Project::CATEGORIES,
            'gradients' => Project::GRADIENTS,
        ]);
    }

    public function update(Request $request, Project $project): RedirectResponse
    {
        $validated = $request->validate($this->rules($project), $this->messages());

        if ($request->hasFile('image')) {
            // replace the old file so the disk does not fill up with orphans
            if ($project->image) {
                Storage::disk('public')->delete($project->image);
            }
            $validated['image'] = $request->file('image')->store('projects', 'public');
        } else {
            // no new upload -> keep the existing image path untouched
            unset($validated['image']);
        }
```

```
        $project->update($validated);

        return redirect()->route('projects.index')
            ->with('success', 'Project updated successfully.');
    }

    public function destroy(Project $project): RedirectResponse
    {
        if ($project->image) {
            Storage::disk('public')->delete($project->image);
        }

        $project->delete();

        return redirect()->route('projects.index')
            ->with('success', 'Project deleted successfully.');
    }

    /**
     * Validation rules shared by store() and update().
     * On update, unique() ignores the record being edited so saving
     * without changing the title does not fail.
     */
    private function rules(?Project $project = null): array
    {
        return [
            'title' => [
                'required', 'string', 'min:3', 'max:100',
                Rule::unique('projects', 'title')->ignore($project?->id),
            ],
            'category' => ['required', Rule::in(Project::CATEGORIES)],
            'gradient' => ['required', Rule::in(Project::GRADIENTS)],
            'description' => ['required', 'string', 'min:20', 'max:1000'],
            'tech_stack' => ['required', 'string', 'max:150'],
            'project_url' => ['nullable', 'url', 'max:255'],
            'repository_url' => ['nullable', 'url', 'max:255'],
            'image' => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
            'date_completed' => ['required', 'date', 'before_or_equal:today'],
            'display_order' => ['required', 'integer', 'between:1,999'],
        ];
    }

    private function messages(): array
    {
        return [
            'title.required' => 'Every project needs a title.',
            'title.unique' => 'A project with this title already exists. Please choose a different title.',
            'category.required' => 'Please choose a category.',
            'category.in' => 'Please pick one of the categories from the list.',
            'description.min' => 'Describe the project in at least 20 characters.',
            'tech_stack.required' => 'List at least one technology (comma-separated).',
            'project_url.url' => 'The project URL must be a valid link, e.g. https://example.com.',
            'repository_url.url' => 'The repository URL must be a valid link, e.g. https://github.com/you/repo.',
            'image.image' => 'The uploaded file must be an image.',
            'image.mimes' => 'Only JPG, PNG or WEBP images are allowed.',
            'image.max' => 'The image is too large. Maximum size is 2 MB.',
            'date_completed.before_or_equal' => 'The completion date cannot be in the future.',
            'display_order.between' => 'Display order must be a number from 1 to 999.',
        ];
    }
}
```

D. Web Routes

routes/web.php

I define the public portfolio route separately from the protected project-management routes. The resource route gives the React pages named CRUD endpoints and requires authentication.

```
<?php

use App\Http\Controllers\ContactController;
use App\Http\Controllers\PortfolioController;
use App\Http\Controllers\ProfileController;
use App\Http\Controllers\ProjectController;
use Illuminate\Support\Facades\Route;
use Inertia\Inertia;

// Public portfolio
Route::get('/', PortfolioController::class)->name('portfolio');
Route::get('/work/{project}', [PortfolioController::class, 'show'])->name('work.show');

// Public contact form (max 5 messages per minute per visitor)
Route::post('/contact', [ContactController::class, 'store'])->middleware('throttle:5,1')->name('contact.store');

// Management pages (login required). Route::resource creates 7 routes;
// show is excluded because the public page already displays the records.
Route::middleware('auth')->group(function () {
    Route::resource('projects', ProjectController::class)->except(['show']);
});

// Breeze defaults
Route::get('/dashboard', function () {
    return Inertia::render('Dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
});

require __DIR__.'/auth.php';
```

E1. React Index Page

resources/js/Pages/Projects/Index.jsx

I use this page to display, search, and paginate the project records. It also opens a confirmation dialog before sending a delete request.

```
import { Head, Link, router } from '@inertiajs/react';
import { useMemo, useState } from 'react';
import SiteLayout from '@/Layouts/SiteLayout';
import ConfirmModal from '@/Components/ConfirmModal';

const rowsPerPage = 5;

export default function Index({ projects }) {
    const [search, setSearch] = useState('');
    const [page, setPage] = useState(1);
    const [toDelete, setToDelete] = useState(null);
    const [deleting, setDeleting] = useState(false);

    const filteredProjects = useMemo(() => {
        const query = search.trim().toLocaleLowerCase();

        if (!query) {
            return projects;
        }

        return projects.filter((project) => {
            const searchable = [
                project.title,
                project.category,
                project.tech_stack,
                ...(project.tech_list ?? []),
            ];

            return searchable.some((value) => String(value ?? '').toLocaleLowerCase().includes(query));
        });
    }, [projects, search]);

    const pageCount = Math.ceil(filteredProjects.length / rowsPerPage);
    const currentPage = Math.min(page, Math.max(pageCount, 1));
    const visibleProjects = filteredProjects.slice(
        (currentPage - 1) * rowsPerPage,
        currentPage * rowsPerPage,
    );

    const confirmDelete = () => {
        setDeleting(true);
        router.delete(route('projects.destroy', toDelete.id), {
            preserveScroll: true,
            onFinish: () => {
                setDeleting(false);
                setToDelete(null);
            },
        });
    };

    return (
        <SiteLayout>
            <Head title="Manage Projects" />

            <div className="mx-auto max-w-6xl px-5 pb-20 pt-32">
                <div className="hero-in mb-8 flex flex-wrap items-end justify-between gap-4">
                    <div>
                        <p className="eyebrow mb-3 text-[var(--terra)]">Admin</p>
                        <h1 className="font-display text-4xl font-bold">Manage <span className="accent">Projects</span></h1>
                    </div>
                    <Link href={route('projects.create')} className="btn">+ New project</Link>
                </div>

                <form onSubmit={(e) => e.preventDefault()} className="mb-6 flex flex-wrap gap-3">
                    <input type="text" value={search} onChange={(e) => { setSearch(e.target.value); setPage(1); }}
placeholder="Search title, category or tech…" className="input max-w-sm" autoComplete="off" />
                    {search && <button type="button" onClick={() => { setSearch(''); setPage(1); }} className="btn-
ghost">Clear</button>}
                </form>

                <div className="card min-h-[22rem] overflow-hidden p-0">
                    <div className="overflow-x-auto">
                        <table className="table">
```

```
                            <colgroup>
                                <col style={{ width: '3.5rem' }} />
                                <col style={{ width: '6.5rem' }} />
                                <col style={{ width: '14rem' }} />
                                <col style={{ width: '9.5rem' }} />
                                <col style={{ minWidth: '12rem' }} />
                                <col style={{ width: '8.5rem' }} />
                                <col style={{ width: '14rem' }} />
                            </colgroup>
                            <thead>
                                <tr>
                                    <th>#</th>
                                    <th>Preview</th>
                                    <th>Title</th>
                                    <th>Category</th>
                                    <th>Tags</th>
                                    <th>Completed</th>
                                    <th>Actions</th>
                                </tr>
                            </thead>
                            <tbody>
                                {filteredProjects.length === 0 && (
                                    <tr><td colSpan={7} className="muted py-12 text-center">No projects found.</td></tr>
                                )}
                                {visibleProjects.map((p) => (
                                    <tr key={p.id}>
                                        <td className="muted">{p.display_order}</td>
                                        <td>
                                            {p.image_url ? (
                                                <img src={p.image_url} alt="" className="h-11 w-16 rounded-lg object-cover" />
                                            ) : (
                                                <div className={`h-11 w-16 rounded-lg grad-${p.gradient}`} />
                                            )}
                                        </td>
                                        <td className="font-display font-bold break-words">{p.title}</td>
                                        <td><span className="chip">{p.category}</span></td>
                                        <td>
                                            <div className="flex max-w-[14rem] flex-wrap gap-1">
                                                {p.tech_list.map((t) => <span key={t} className="chip">{t}</span>)}
                                            </div>
                                        </td>
                                        <td className="whitespace-nowrap muted">{p.date_completed}</td>
                                        <td className="whitespace-nowrap">
                                            <div className="actions-buttons">
                                                <Link href={route('projects.edit', p.id)} className="btn-ghost btn-
sm">Edit</Link>

                                                <button type="button" onClick={() => setToDelete(p)} className="btn-danger btn-
sm">Delete</button>

                                            </div>
                                        </td>
                                    </tr>
                                ))}
                            </tbody>
                        </table>
                    </div>
                </div>

                {pageCount > 1 && (
                    <nav className="mt-8 flex flex-wrap justify-center gap-2" aria-label="Pagination">
                        <button type="button" onClick={() => setPage(currentPage - 1)} disabled={currentPage === 1} className=
{`${currentPage === 1 ? 'btn-ghost opacity-40' : 'btn-ghost'} btn-sm`}>Previous</button>
                        {Array.from({ length: pageCount }, (_, index) => index + 1).map((pageNumber) => (
                            <button key={pageNumber} type="button" onClick={() => setPage(pageNumber)} aria-current=
{currentPage === pageNumber ? 'page' : undefined} className={`${currentPage === pageNumber ? 'btn' : 'btn-ghost'} btn-sm`}>
                                {pageNumber}
                            </button>
                        ))}
                        <button type="button" onClick={() => setPage(currentPage + 1)} disabled={currentPage === pageCount}
className={`${currentPage === pageCount ? 'btn-ghost opacity-40' : 'btn-ghost'} btn-sm`}>Next</button>
                    </nav>
                )}
            </div>

            <ConfirmModal
                show={toDelete !== null}
                title="Delete this project?"
                message={toDelete ? `"${toDelete.title}" will be removed from your portfolio permanently. This cannot be
undone.` : ''}
                processing={deleting}
                onConfirm={confirmDelete}
                onCancel={() => setToDelete(null)}
            />
        </SiteLayout>
    );
}
```

E2. React Create Page

resources/js/Pages/Projects/Create.jsx

I use the create page to initialize an empty Inertia form and pass it to the shared project form. On submit, it sends multipart form data so an optional image can be uploaded.

```
import { Head, useForm } from '@inertiajs/react';
import SiteLayout from '@/Layouts/SiteLayout';
import ProjectForm from '@/Components/ProjectForm';

export default function Create({ categories, gradients, nextOrder = 1 }) {
    const { data, setData, post, processing, errors } = useForm({
        title: '',
        category: categories?.[0] ?? 'UI Design',
        gradient: 'green',
        description: '',
        tech_stack: '',
        project_url: '',
        repository_url: '',
        image: null,
        date_completed: '',
        display_order: nextOrder,
    });

    const submit = (e) => {
        e.preventDefault();
        // forceFormData sends the image as multipart/form-data.
        // If validation fails, Inertia keeps `data` and fills `errors` - nothing typed is lost.
        post(route('projects.store'), { forceFormData: true });
    };

    return (
        <SiteLayout>
            <Head title="New Project" />
            <div className="mx-auto max-w-2xl px-5 pb-20 pt-32">
                <p className="hero-in eyebrow mb-3 text-[var(--terra)]">Admin / Projects</p>
                <h1 className="hero-in font-display mb-8 text-4xl font-bold" style={{ '--d': '100ms' }}>New <span
className="accent">Project</span></h1>
                <div className="card hero-in p-6 sm:p-9" style={{ '--d': '200ms' }}>
                    <ProjectForm data={data} setData={setData} errors={errors} processing={processing} onSubmit={submit}
submitLabel="Create project" categories={categories} gradients={gradients} />
                </div>
            </div>
        </SiteLayout>
    );
}
```

E3. React Edit Page

resources/js/Pages/Projects/Edit.jsx

I use the edit page to populate the same shared form with the selected project. It uses method spoofing for the file upload and preserves the existing image when no replacement is chosen.

```
import { Head, useForm } from '@inertiajs/react';
import SiteLayout from '@/Layouts/SiteLayout';
import ProjectForm from '@/Components/ProjectForm';

export default function Edit({ project, categories, gradients }) {
    const { data, setData, post, processing, errors } = useForm({
        _method: 'put', // files cannot be sent with a real PUT, so we POST and Laravel treats it as PUT
        title: project.title,
        category: project.category,
        gradient: project.gradient,
        description: project.description,
        tech_stack: project.tech_stack,
        project_url: project.project_url ?? '',
        repository_url: project.repository_url ?? '',
        image: null, // stays null unless a new file is chosen -> the old image is kept
        date_completed: project.date_completed,
        display_order: project.display_order,
    });

    const submit = (e) => {
        e.preventDefault();
        post(route('projects.update', project.id), { forceFormData: true });
    };

    return (
        <SiteLayout>
            <Head title={`Edit ${project.title}`} />
            <div className="mx-auto max-w-2xl px-5 pb-20 pt-32">
                <p className="hero-in eyebrow mb-3 text-[var(--terra)]">Admin / Projects / {project.id}</p>
                <h1 className="hero-in font-display mb-8 text-4xl font-bold" style={{ '--d': '100ms' }}>Edit <span
className="accent">Project</span></h1>
                <div className="card hero-in p-6 sm:p-9" style={{ '--d': '200ms' }}>
                    <ProjectForm
                        data={data}
                        setData={setData}
                        errors={errors}
                        processing={processing}
                        onSubmit={submit}
                        submitLabel="Save changes"
                        existingImageUrl={project.image_url}
                        categories={categories}
                        gradients={gradients}
                    />
                </div>
            </div>
        </SiteLayout>
    );
}
```

E4. Public Portfolio Page

resources/js/Pages/Portfolio.jsx

I use this public page to render the Projects section from database records supplied by the portfolio controller. The layout adapts the project cards and scrolling controls to the number of records.

```
import { Head, Link, usePage } from '@inertiajs/react';
import { useEffect, useRef, useState } from 'react';
import SiteLayout from '@/Layouts/SiteLayout';
import SectionTitle from '@/Components/SectionTitle';
import Card from '@/Components/Card';
import ProjectCard from '@/Components/ProjectCard';
import ContactForm from '@/Components/ContactForm';
import Reveal from '@/Components/Reveal';
import { PROFILE, SKILLS, TOOLS } from '@/data/profile';

export default function Portfolio({ projects }) {
    const { auth } = usePage().props;
    const [photoOk, setPhotoOk] = useState(true);
    const projectScrollerRef = useRef(null);
    const [canScrollProjectsLeft, setCanScrollProjectsLeft] = useState(false);
    const [canScrollProjectsRight, setCanScrollProjectsRight] = useState(false);

    useEffect(() => {
        const scroller = projectScrollerRef.current;
        if (!scroller || projects.length < 4) return undefined;

        scroller.scrollLeft = 0;

        const updateScrollButtons = () => {
            setCanScrollProjectsLeft(scroller.scrollLeft > 1);
            setCanScrollProjectsRight(scroller.scrollWidth - scroller.clientWidth - scroller.scrollLeft > 1);
        };

        updateScrollButtons();
        scroller.addEventListener('scroll', updateScrollButtons, { passive: true });
        window.addEventListener('resize', updateScrollButtons);

        return () => {
            scroller.removeEventListener('scroll', updateScrollButtons);
            window.removeEventListener('resize', updateScrollButtons);
        };
    }, [projects.length]);

    const scrollProjects = (direction) => {
        const scroller = projectScrollerRef.current;
        if (!scroller) return;

        const card = scroller.querySelector('.project-card-item');
        const cardWidth = card?.getBoundingClientRect().width ?? 352;
        scroller.scrollBy({ left: direction * (cardWidth + 28), behavior: 'smooth' });
    };

    return (
        <SiteLayout>
            <Head title={`${PROFILE.name} | Designer & Aspiring Data Analyst`} />

            {/* ————————— HERO ————————— */}
            <section className="relative flex min-h-screen items-center overflow-hidden pt-20">
                <div className="blob palette-blob-sage -right-24 top-24 h-[26rem] w-[26rem]" />
                <div className="blob palette-blob-light-sage -bottom-20 right-1/4 h-80 w-80" style={{ animationDelay: '-4s' }}
/>
                <div className="blob palette-blob-cream left-[-6rem] top-1/3 h-64 w-64" style={{ animationDelay: '-8s' }} />
                <span aria-hidden="true" className="clay-shape clay-sphere hero-clay-sphere" />
                <span aria-hidden="true" className="clay-shape clay-cube hero-clay-cube" />
                <span aria-hidden="true" className="clay-shape clay-ring hero-clay-ring" />

                <div className="relative mx-auto w-full max-w-6xl px-5 py-16">
                    <div className="hero-in mb-8 inline-flex items-center gap-3 rounded-full bg-[var(--sand)] px-5 py-2.5 text-
sm font-medium" style={{ '--d': '100ms' }}>
                        <span className="pulse-dot" />
                        {PROFILE.status} ✦
                    </div>

                    <h1 className="font-display text-[2.6rem] font-bold leading-[1.08] sm:text-6xl lg:text-[5.4rem]">
                        <span className="hero-in block" style={{ '--d': '250ms' }}>
                            Hi, I'm <span className="accent draw-underline">{PROFILE.name}</span>
                        </span>
                        <span className="hero-in block" style={{ '--d': '400ms' }}>Designer</span>
```

```
                        <span className="hero-in block" style={{ '--d': '550ms' }}>&amp;amp; Aspiring Data Analyst.</span>
                    </h1>

                    <p className="hero-in muted mt-8 max-w-xl text-lg leading-relaxed" style={{ '--d': '750ms' }}>
{PROFILE.intro}</p>

                    <div className="hero-in mt-10 flex flex-wrap gap-4" style={{ '--d': '900ms' }}>
                        <a href="#projects" className="btn">View Work</a>
                        <a href="#contact" className="btn-outline">Get in Touch</a>
                    </div>

                    <a href="#about" className="hero-in muted mt-14 inline-flex items-center gap-2 text-sm tracking-widest"
style={{ '--d': '1100ms' }}>
                        <span className="bob">↓</span> scroll
                    </a>
                </div>
            </section>

            {/* ————— ABOUT ————— */}
            <section id="about" className="clay-section py-24">
                <div className="mx-auto grid max-w-6xl items-center gap-14 px-5 md:grid-cols-[1fr_1.15fr]">
                    <Reveal variant="left" className="relative mx-auto w-full max-w-[20rem] md:max-w-[28rem]">
                        <div className="clay-photo about-photo-frame" style={{ aspectRatio: '892 / 958' }}>
                            {photoOk ? (
                                <img src={PROFILE.photo} alt={PROFILE.name} onError={() => setPhotoOk(false)} className="about-
photo-image" />
                            ) : (
                                <div className="clay-photo-fallback font-display flex h-full w-full items-center justify-center
text-8xl">NP</div>
                            )}
                        </div>
                        <div className="badge-float absolute -bottom-5 -right-3 rounded-2xl bg-[var(--sand)] px-5 py-3 text-sm
font-bold shadow-lg sm:-right-6">
                            No exp.
                        </div>
                    </Reveal>

                    <div>
                        <Reveal><p className="eyebrow muted mb-4">About me</p></Reveal>
                        <Reveal delay={100}>
                            <h2 className="font-display text-4xl font-bold leading-tight sm:text-5xl">
                                {PROFILE.aboutTitle[0]} <span className="accent">{PROFILE.aboutTitle[1]}</span>
{PROFILE.aboutTitle[2]}
                            </h2>
                        </Reveal>
                        {PROFILE.about.map((p, i) => (
                            <Reveal key={i} delay={200 + i * 100}>
                                <p className="muted mt-6 text-[1.02rem] leading-[1.85]">{p}</p>
                            </Reveal>
                        ))}
                        <ul className="mt-8 space-y-4">
                            {PROFILE.facts.map((f, i) => (
                                <Reveal as="li" key={f.text} delay={450 + i * 100} className="fact-row flex items-center gap-
4">
                                    <span className="text-xl">{f.icon}</span>
                                    <span className="font-medium">{f.text}</span>
                                </Reveal>
                            ))}
                        </ul>
                    </div>
                </div>
            </section>

            {/* ————— SKILLS ————— */}
            <section id="skills" className="palette-skills-section py-24">
                <div className="mx-auto max-w-6xl px-5">
                    <SectionTitle eyebrow="What I know" accent title="Skills & Tools" />

                    <div className="grid gap-6 sm:grid-cols-2 lg:grid-cols-4">
                        {SKILLS.map((s, i) => (
                            <Reveal key={s.title} delay={i * 120} variant="zoom" className="h-full">
                                <Card hover className="skill-card flex h-full flex-col p-7" tabIndex={0}>
                                    <div className="skill-emoji mb-6 text-4xl">{s.icon}</div>
                                    <h3 className="font-display mb-3 text-xl font-bold">{s.title}</h3>
                                    <p className="muted mb-7 flex-1 text-[.95rem] leading-relaxed">{s.text}</p>
                                    <div className="mt-auto">
                                        <div className="mb-2 flex justify-end">
                                            <span className="skill-percentage" style={{ '--skill-level': s.level }} aria-label=
{`${s.level}%`} />
                                        </div>
                                        <div
                                            className="skill-bar"
                                            style={{ '--w': `${s.level}%` }}
                                            role="progressbar"
                                            aria-label={`${s.title} proficiency`}
                                            aria-valuemin={0}
```

```
                                        aria-valuemax={100}
                                        aria-valuenow={s.level}
                                    >
                                        <span />
                                    </div>
                                </div>
                            </Card>
                        </Reveal>
                    ))}
                </div>

                <Reveal delay={200} className="marquee mt-14">
                    <div className="marquee-track">
                        {[...TOOLS, ...TOOLS].map((t, i) => (
                            <span key={i} className="chip whitespace-nowrap px-6 py-2.5 text-sm">{t}</span>
                        ))}
                    </div>
                </Reveal>
            </div>
        </section>

        {/* ————— PROJECTS (from the database) ————— */}
        <section id="projects" className="clay-section scroll-mt-28 py-24">
            <div className="mx-auto max-w-6xl px-5">
                <div className={`project-heading ${projects.length >= 4 ? 'has-controls' : ''}`}>
                    <Reveal className="project-heading-title text-center">
                        <p className="eyebrow mb-4 text-[var(--terra)]">My work</p>
                        <h2 className="font-display text-4xl font-bold leading-tight sm:text-5xl">Featured Projects</h2>
                    </Reveal>
                    {projects.length >= 4 && (
                        <div className="project-heading-controls flex justify-center gap-3">
                            <button
                                type="button"
                                onClick={() => scrollProjects(-1)}
                                disabled={!canScrollProjectsLeft}
                                aria-label="Scroll projects left"
                                className="project-scroll-button"
                            >
                                ←
                            </button>
                            <button
                                type="button"
                                onClick={() => scrollProjects(1)}
                                disabled={!canScrollProjectsRight}
                                aria-label="Scroll projects right"
                                className="project-scroll-button"
                            >
                                →
                            </button>
                        </div>
                    )}
                </div>

                {projects.length === 0 ? (
                    <p className="muted text-center">
                        No projects yet.
                        {auth?.user && <> <Link href={route('projects.create')} className="text-[var(--terra)]
underline">Add your first project</Link></>}
                    </p>
                ) : (
                    <div
                        ref={projectScrollerRef}
                        className={`projects-row ${projects.length >= 4 ? 'projects-row--scroll' : 'projects-row--
center'}`}
                    >
                        {projects.map((p, i) => (
                            <Reveal key={p.id} delay={(i % 3) * 130} className="project-card-item">
                                <ProjectCard project={p} />
                            </Reveal>
                        ))}
                    </div>
                )}
            </div>
        </section>

        {/* ————— CONTACT ————— */}
        <section id="contact" className="relative py-24">
            <span aria-hidden="true" className="clay-shape clay-sphere contact-clay-sphere" />
            <span aria-hidden="true" className="clay-shape clay-cube contact-clay-cube" />
            <div className="mx-auto max-w-2xl px-5">
                <SectionTitle
                    title="Let's Work Together"
                    subtitle="Have a project in mind or just want to connect? Fill out the form below or shoot me an
email."
                />
                <Reveal delay={100}>
```

```
                        <Card className="p-6 sm:p-9">
                            <ContactForm />
                        </Card>
                        <p className="muted mt-6 text-center text-sm">
                            or email me directly at <a href={`mailto:${PROFILE.email}`} className="text-[var(--terra)]
underline-offset-4 hover:underline">{PROFILE.email}</a>
                        </p>
                    </Reveal>
                </div>
            </section>
        </SiteLayout>
    );
}
```

F1. Shared Project Form

resources/js/Components/ProjectForm.jsx

I use one form component for both create and edit so the fields, labels, and inline errors stay consistent. The parent pages keep ownership of form state and submission behavior.

```
import { Link } from '@inertiajs/react';
import Field from '@/Components/Field';
import { CATEGORIES, GRADIENTS } from '@/data/profile';

// ONE form used by BOTH Create and Edit. The page owns the data (useForm)
// and passes it in; this component only renders the inputs and the errors.
export default function ProjectForm({ data, setData, errors, processing, onSubmit, submitLabel, existingImageUrl = null,
categories = CATEGORIES, gradients = GRADIENTS }) {
    const cls = (name) => `input ${errors[name] ? 'has-error' : ''}`;

    return (
        <form onSubmit={onSubmit} className="space-y-6" noValidate>
            <Field label="Title" htmlFor="title" error={errors.title} required>
                <input id="title" type="text" className={cls('title')} value={data.title} onChange={(e) => setData('title',
e.target.value)} placeholder="School Student Portal" />
            </Field>

            <div className="grid gap-6 sm:grid-cols-2">
                <Field label="Category" htmlFor="category" error={errors.category} required>
                    <select id="category" className={cls('category')} value={data.category} onChange={(e) =>
setData('category', e.target.value)}>
                        {categories.map((c) => <option key={c}>{c}</option>)}
                    </select>
                </Field>
                <Field label="Card colour" htmlFor="gradient" error={errors.gradient} hint="Shown when there is no image.">
                    <select id="gradient" className={cls('gradient')} value={data.gradient} onChange={(e) =>
setData('gradient', e.target.value)}>
                        {gradients.map((g) => <option key={g} value={g}>{g}</option>)}
                    </select>
                </Field>
            </div>

            <Field label="Description" htmlFor="description" error={errors.description} hint="20 to 1000 characters." required>
                <textarea id="description" rows={4} className={cls('description')} value={data.description} onChange={(e) =>
setData('description', e.target.value)} placeholder="What is the project about?" />
            </Field>

            <Field label="Tech stack / tags" htmlFor="tech_stack" error={errors.tech_stack} hint="Separate with commas."
required>
                <input id="tech_stack" type="text" className={cls('tech_stack')} value={data.tech_stack} onChange={(e) =>
setData('tech_stack', e.target.value)} placeholder="UI Design, Figma, Web" />
            </Field>

            <div className="grid gap-6 sm:grid-cols-2">
                <Field label="Project URL" htmlFor="project_url" error={errors.project_url}>
                    <input id="project_url" type="text" className={cls('project_url')} value={data.project_url} onChange={(e)
=> setData('project_url', e.target.value)} placeholder="https://example.com" />
                </Field>
                <Field label="Repository URL" htmlFor="repository_url" error={errors.repository_url}>
                    <input id="repository_url" type="text" className={cls('repository_url')} value={data.repository_url}
onChange={(e) => setData('repository_url', e.target.value)} placeholder="https://github.com/you/repo" />
                </Field>
            </div>

            <div className="grid gap-6 sm:grid-cols-2">
                <Field label="Date completed" htmlFor="date_completed" error={errors.date_completed} required>
                    <input id="date_completed" type="date" className={cls('date_completed')} value={data.date_completed}
onChange={(e) => setData('date_completed', e.target.value)} />
                </Field>
                <Field label="Display order" htmlFor="display_order" error={errors.display_order} hint="1 appears first on the
public page." required>
                    <input id="display_order" type="number" className={cls('display_order')} value={data.display_order}
onChange={(e) => setData('display_order', e.target.value)} />
                </Field>
            </div>

            <Field label="Image" htmlFor="image" error={errors.image} hint="JPG, PNG or WEBP, max 2 MB. Leave empty to keep the
current one.">
                {existingImageUrl && <img src={existingImageUrl} alt="Current project" className="mb-3 h-28 rounded-xl border
border-[var(--line)] object-cover" />}
                <input id="image" type="file" accept="image/*" className={cls('image')} onChange={(e) => setData('image',
e.target.files[0] ?? null)} />
            </Field>
```

```
            <div className="flex flex-wrap items-center gap-3 pt-2">
                <button type="submit" className="btn" disabled={processing}>{processing ? 'Saving...' : submitLabel}</button>
                <Link href={route('projects.index')} className="btn-ghost">Cancel</Link>
            </div>
        </form>
    );
}
```

F2. Field Component

resources/js/Components/Field.jsx

I use this small component to keep field labels, hints, and validation messages consistent. It exposes validation messages with an alert role for assistive technology.

```
// Label + input slot + validation error beside it, so every field looks and behaves the same.
export default function Field({ label, htmlFor, error, hint, required = false, children }) {
    return (
        <div>
            <label htmlFor={htmlFor} className="label">
                {label}
                {required && <span className="text-[var(--terra)]"> *</span>}
            </label>
            {children}
            {hint && !error && <p className="muted mt-1.5 text-xs">{hint}</p>}
            {error && <p className="error" role="alert">{error}</p>}
        </div>
    );
}
```

F3. Delete Confirmation

resources/js/Components/ConfirmModal.jsx

I use this modal to make the irreversible delete action explicit before it is sent. It also disables the controls while the request is processing.

```
// Confirmation step shown before a record is deleted.
export default function ConfirmModal({ show, title, message, confirmLabel = 'Delete', processing = false, onConfirm, onCancel
}) {
    if (!show) return null;

    return (
        <div className="fixed inset-0 z-[80] flex items-center justify-center bg-[#1f2d22]/50 p-4 backdrop-blur-sm"
role="dialog" aria-modal="true">
            <div className="modal w-full max-w-md p-7">
                <div className="modal-icon mb-4 flex h-12 w-12 items-center justify-center rounded-full text-xl">⚠</div>
                <h3 className="font-display mb-2 text-2xl font-bold">{title}</h3>
                <p className="muted mb-7 text-[.95rem] leading-relaxed">{message}</p>
                <div className="flex justify-end gap-3">
                    <button type="button" className="btn-ghost" onClick={onCancel} disabled={processing}>Cancel</button>
                    <button type="button" className="btn-danger" onClick={onConfirm} disabled={processing}>
                        {processing ? 'Deleting...' : confirmLabel}
                    </button>
                </div>
            </div>
        </div>
    );
}
```

F4. Success Message

resources/js/Components/FlashMessage.jsx

I use this component to show one-time success feedback returned by Laravel after a CRUD operation. The notification dismisses itself after a short delay.

```
import { usePage } from '@inertiajs/react';
import { useEffect, useState } from 'react';

// Shows the one-time "success" message sent by the controller, then hides itself.
export default function FlashMessage() {
    const { flash } = usePage().props;
    const [visible, setVisible] = useState(false);

    useEffect(() => {
        if (!flash?.success) return;
        setVisible(true);
        const timer = setTimeout(() => setVisible(false), 4500);
        return () => clearTimeout(timer);
    }, [flash]);

    if (!visible || !flash?.success) return null;

    return (
        <div role="status" className="toast fixed right-4 top-24 z-[70] flex max-w-sm items-start gap-3 rounded-2xl px-5 py-4
text-sm">
            <span className="toast-icon mt-0.5 flex h-5 w-5 shrink-0 items-center justify-center rounded-full text-
xs">✓</span>
            <p className="flex-1">{flash.success}</p>
            <button type="button" onClick={() => setVisible(false)} className="muted hover:text-[var(--ink)]" aria-
label="Dismiss">×</button>
        </div>
    );
}
```

F5. Public Project Card

resources/js/Components/ProjectCard.jsx

I use this component to present one project consistently in the public portfolio. It displays the project image or fallback gradient, category, description, and technology tags.

```
import { Link } from '@inertiajs/react';
import Card from '@/Components/Card';

// One project on the public portfolio. Uses the uploaded image, or a colour gradient when there is none.
export default function ProjectCard({ project }) {
    return (
        <Link
            href={route('work.show', project.id)}
            aria-label={`View ${project.title}`}
            className="project-card-link block h-full w-full rounded-[2rem] focus-visible:outline focus-visible:outline-2
focus-visible:outline-offset-4 focus-visible:outline-[var(--terra)]"
        >
            <Card hover className="project-card-root flex h-full w-full min-w-0 flex-col overflow-hidden !p-0">
                <div className="project-image zoom-wrap relative">
                    {project.image_url ? (
                        <img src={project.image_url} alt="" className="h-full w-full object-cover" loading="lazy" />
                    ) : (
                        <div className={`h-full w-full grad-${project.gradient}`} />
                    )}
                    <span className="clay-project-badge absolute bottom-4 left-4 rounded-full px-4 py-1.5 text-xs font-
extrabold uppercase tracking-widest backdrop-blur">
                        {project.category}
                    </span>
                    <span aria-hidden="true" className="project-card-cue">
                        View project →
                    </span>
                </div>

                <div className="project-card-content flex min-w-0 flex-col p-6">
                    <h3 className="font-display mb-3 line-clamp-2 text-xl font-bold">{project.title}</h3>
                    <p className="muted line-clamp-3 text-[.95rem] leading-relaxed">{project.description}</p>
                    <div className="project-card-bottom">
                        <div className="project-card-tags flex flex-wrap items-center gap-2 overflow-hidden">
                            {project.tech_list.map((t) => (
                                <span key={t} className="chip">{t}</span>
                            ))}
                        </div>
                    </div>
                </div>
            </Card>
        </Link>
    );
}
```

F6. Navbar Component

resources/js/Components/Navbar.jsx

I use the navbar to provide consistent navigation between the portfolio sections and pages. On the home page it tracks the active section and includes a responsive mobile menu.

```
import { Link, usePage } from '@inertiajs/react';
import { useEffect, useState } from 'react';
import { PROFILE } from '@/data/profile';

const LINKS = ['about', 'skills', 'projects', 'contact'];

export default function Navbar() {
    const { component } = usePage();
    const onHome = component === 'Portfolio';
    const [open, setOpen] = useState(false);
    const [scrolled, setScrolled] = useState(false);
    const [progress, setProgress] = useState(0);
    const [active, setActive] = useState('');

    // shadow + progress bar while scrolling
    useEffect(() => {
        const onScroll = () => {
            const max = document.documentElement.scrollHeight - window.innerHeight;
            setScrolled(window.scrollY > 10);
            setProgress(max > 0 ? (window.scrollY / max) * 100 : 0);
        };
        onScroll();
        window.addEventListener('scroll', onScroll, { passive: true });
        return () => window.removeEventListener('scroll', onScroll);
    }, []);

    // highlight the link of the section currently on screen (home page only)
    useEffect(() => {
        if (!onHome) return;
        const io = new IntersectionObserver(
            (entries) => entries.forEach((e) => e.isIntersecting && setActive(e.target.id)),
            { rootMargin: '-45% 0px -50% 0px' }
        );
        LINKS.forEach((id) => {
            const el = document.getElementById(id);
            if (el) io.observe(el);
        });
        return () => io.disconnect();
    }, [onHome]);

    const href = (id) => (onHome ? `#${id}` : `${route('portfolio')}#${id}`);

    return (
        <header className={`clay-navbar fixed inset-x-0 top-0 z-50 transition-all duration-300 ${scrolled ? 'backdrop-blur' :
'backdrop-blur-sm'}`}>
            <div className="clay-progress absolute bottom-0 left-0 h-[2px] transition-[width] duration-100" style={{ width:
`${progress}%` }} />
            <nav className="mx-auto flex h-16 max-w-6xl items-center justify-between px-5 sm:h-20">
                <Link href={route('portfolio')} className="font-display text-2xl font-bold">
                    {PROFILE.firstName}<span className="text-[var(--terra)]">.</span>
                </Link>

                <ul className="hidden items-center gap-9 text-[.95rem] md:flex">
                    {LINKS.map((id) => (
                        <li key={id}>
                            <a href={href(id)} className={`nav-link capitalize ${active === id ? 'active' : ''}`}>{id}</a>
                        </li>
                    ))}
                </ul>

                <button
                    type="button"
                    onClick={() => setOpen(!open)}
                    className="clay-menu-toggle relative h-10 w-10 md:hidden"
                    aria-label="Toggle menu"
                    aria-expanded={open}
                >
                    <span className={`absolute left-2 h-0.5 w-6 bg-[var(--ink)] transition-all duration-300 ${open ? 'top-5
rotate-45' : 'top-3'}`} />
                    <span className={`absolute left-2 top-5 h-0.5 w-6 bg-[var(--ink)] transition-opacity duration-200 ${open ?
'opacity-0' : ''}`} />
                    <span className={`absolute left-2 h-0.5 w-6 bg-[var(--ink)] transition-all duration-300 ${open ? 'top-5 -
rotate-45' : 'top-7'}`} />
```

```
                </button>
            </nav>

            {/* mobile drawer */}
            <div className={`clay-drawer overflow-hidden transition-[max-height] duration-300 md:hidden ${open ? 'max-h-80' :
'max-h-0'}`}>
                <ul className="flex flex-col gap-1 p-5">
                    {LINKS.map((id) => (
                        <li key={id}>
                            <a href={href(id)} onClick={() => setOpen(false)} className="block rounded-xl px-3 py-3 capitalize
hover:bg-white">{id}</a>
                        </li>
                    ))}
                </ul>
            </div>
        </header>
    );
}
```

5. Design Decisions

a. Why use a separate resource controller?
I use a dedicated ProjectController to keep project CRUD actions together and separate from the public portfolio and contact features. Laravel's resource routing maps the standard create, read, update, and delete actions to clear named routes.

b. Why these validation rules, and where?
I validate incoming data on the server in ProjectController so invalid requests cannot bypass browser controls. Required fields, allowed categories/gradients, uniqueness, URL formats, image type/size, a non-future completion date, and a bounded display order reflect the data the project records need; custom messages explain common errors near the fields.

c. How are React pages and components organized?
I group management pages under `Pages/Projects` and keep the public portfolio page separate. I reuse ProjectForm and small display components so create/edit fields, errors, project cards, feedback, and confirmations stay consistent.

6. Validation Test Table

> **Draft based on the implemented rules:** The original Part 4 table was not supplied. Only the blank-required-fields case and a valid create/delete screenshot flow were exercised for this report; other rows describe expected rule outcomes, not individually executed tests.

#	Input / scenario	Expected result	Report evidence
1	Leave title, description, tech stack, and completion date empty.	Reject and show required-field messages.	Observed in Figure 7.
2	Title shorter than 3 characters or longer than 100.	Reject under title length rules.	Rule present; not individually run.
3	Use a title already assigned to another project.	Reject with custom duplicate-title message.	Rule present; not individually run.
4	Description shorter than 20 or longer than 1000 characters.	Reject under description length rules.	Rule present; not individually run.
5	Enter malformed project or repository URL.	Reject with URL format validation.	Rule present; not individually run.
6	Upload unsupported or over-2-MB image.	Reject image type/size and show a useful message.	Rule present; not individually run.
7	Set completion date in the future or display order outside 1-999.	Reject date/order and show validation feedback.	Rules present; not individually run.
8	Submit valid project data.	Create record and show success notification.	Observed in Figure 6; temporary record removed after capture.
9	Edit a project without changing its title.	Allow save because uniqueness ignores the current record.	Rule present; not individually run.

7. Reflection

a. Hardest part. **Draft:** The hardest part was connecting the public portfolio to data that could also be safely managed. I kept the public query in the portfolio controller, used a separate authenticated resource controller for changes, and passed the project data into React through Inertia.

b. What I learned about validation. **Draft:** I learned that client-side form feedback is not a substitute for server-side validation. Laravel can validate the submitted data, return field-specific errors through Inertia, preserve the entered values, and use custom messages to make corrections clearer.

c. A second dynamic section. **Draft:** If I added another dynamic section, I would define its schema, validation cases, and reusable UI patterns before building the pages. I would also add focused feature tests for valid submissions, invalid data, authorization, updates, and deletes so the behavior is verified independently of screenshots.

8. Links and Source Codes

GitHub repository: https://github.com/njcp19/AWD

Video demonstration: Not provided. Add the video URL before submitting.

Local validation: Laravel test suite reported 25 passing tests; production frontend build completed successfully.

Prepared from the current project source and local screenshots. Review all items marked draft/not provided and replace them with your own assignment answers before submission.